IUT d’Angoulême

Boubacar DIALLO

Rapport du projet

r

Table des matières

Introduction 2
1 Présentation technique 3
1.1 Le convertisseur : MCP3201 3
1.2 Le microcontrôleur ATTiny85 3
2 Conception et développement 4
2.1 Réalisation d'un support de programmation et test de l'ATTiny 4
2.1.1 En mode programmation 4
2.1.2 En mode utilisation 4
2.2 Explication des composants essentielles du schéma 4
3 Programmation de l'ATTiny 5
3.1 Configuration des broches 5
3.2 Boucle principale 6
3.3 En fonction de la valeur de donne_numerique 7
4 Connexion de l'ensemble (MCP et ATTiny) pour allumer les LEDs 9
5 Compétences développées au cours de cette SAE 10
Conclusion 11

Introduction

Dans le cadre de mes études en Génie Électrique et Informatique Industrielle (GEII), j'ai mené à bien, en autonomie, la conception et la réalisation d'un stylo voltmètre. Ce projet, à la fois compact et fonctionnel, permet de mesurer des tensions continues comprises entre 0 et 14 volts. Cette expérience a été l'occasion d'explorer des compétences clés en électronique et programmation, tout en relevant divers défis techniques liés à la conception et à la mise en œuvre.

Le stylo voltmètre repose sur deux composants électroniques majeurs :

- **Le MCP3201**, un convertisseur analogique-numérique (CAN) à 12 bits, qui garantit une mesure précise des tensions. Cependant, sa plage de mesure limitée à 5 volts a nécessité l'intégration d'un diviseur de tension pour adapter les valeurs d'entrée.
- **L'ATTiny85**, un microcontrôleur compact, chargé de recevoir les données via une interface SPI, de les traiter, et de piloter l'éclairage de LEDs pour afficher visuellement la plage de tension mesurée.

Le fonctionnement du système est structuré comme suit :

- **0 à 2 volts** : une LED verte s'allume.
- **2 à 4 volts** : une LED rouge s'active.
- **4 à 5 volts (ou environ 14 volts après conversion)** : les deux LEDs s'allument simultanément.

Le dispositif effectue une mesure toutes les 10 millisecondes, garantissant une réactivité adaptée à des applications pratiques.

Le fait de travailler seul sur ce projet m'a permis d'acquérir une vue d'ensemble complète, allant de la phase de conception initiale jusqu'aux tests finaux. J'ai débuté par une étude approfondie des composants pour en comprendre les caractéristiques et leur interconnexion. Cette étape a été suivie par la réalisation d'un schéma électronique et la création d'un prototype sur une plaque d'essai.

Cependant, le développement n'a pas été exempt de difficultés. Lors de la programmation de l'ATTiny85 avec l'outil **AVRDUDE**, un problème imprévu est survenu : mon antivirus bloquait le logiciel. Pour contourner cette limitation, j'ai utilisé une **carte Arduino comme programmateur**, ce qui m'a permis de charger le code sur l'ATTiny85 et de poursuivre le projet. Cette solution improvisée m'a offert l'opportunité de consolider mes compétences en programmation embarquée tout en développant une approche adaptable face aux imprévus.

Avant d'entrée dans le vif du sujet, il serait très important d'introduire le MCP3201 et ATTiny85.

1 Présentation technique

1.1 Le convertisseur : MCP3201

Le MCP3201 est un convertisseur analogique-numérique (CAN) de 12 bits à approximations successives. Ses caractéristiques principales sont les suivantes :

- Résolution : 12 bits.
- Plage de tension d'alimentation : 2,7 V à 5,5 V.
- Consommation : 5 nA en veille, 400 µA en activité.
- Interface de communication : SPI (Serial Peripheral Interface).

La liaison SPI nécessite quatre signaux principaux :

- SCLK (Serial Clock) : horloge générée par le maître (ATTiny85).
- MOSI (Master Out Slave In) : données transmises du maître vers l'esclave.
- MISO (Master In Slave Out) : données transmises de l'esclave vers le maître.
- CS (Chip Select) : activation de l'esclave par le maître.

Pour permettre la mesure de tensions jusqu'à 14V, un diviseur de tension est utilisé pour réduire cette plage à 0-5V, compatible avec le MCP3201. Une protection contre les surtensions est ajoutée via une diode Zener et une résistance limitant le courant.

1.2 Le microcontrôleur ATTiny85

Le microcontrôleur ATTiny85 est utilisé pour traiter les données issues du MCP3201 et contrôler les LEDs. Ses caractéristiques principales incluent :

- Architecture : AVR 8 bits.
- Mémoire Flash : 8 Ko.
- Interface SPI : pour communiquer avec le MCP3201.
- Tension d'alimentation : 2,7 V à 5,5 V.

L'ATTiny85 analyse les données transmises par le MCP3201 et allume les LEDs selon les plages de tension mesurées.

2 Conception et développement

Le projet s'est déroulé en trois étapes principales. Tout d'abord, j'ai conçu un support de programmation adapté, nécessaire pour configurer correctement le microcontrôleur ATTiny85. Ensuite, j'ai programmé ce dernier pour qu'il génère les signaux nécessaires, notamment le signal CS (Chip Select) et l'horloge, et qu'il puisse également recevoir les données mesurées. Enfin, j'ai connecté et intégré l'ensemble du système, en reliant le MCP3201 au microcontrôleur pour assurer le fonctionnement complet, avec l'allumage des LEDs en fonction des tensions détectées.

2.1 Réalisation d'un support de programmation et test de l'ATTiny

Cette partie a été réalisée pour permettre la programmation et l'utilisation de l'ATTiny85 dans le cadre du projet. Le routage du circuit a été conçu pour assurer deux modes distincts : programmation et utilisation

2.1.1 En mode programmation

On débranche le cavalier de J4 et celui de J3 pour isoler l'alimentation externe pour éviter des conflits. On branche le programmateur USBasp sur le connecteur J1 (Prog). Le programmateur USBasp fournit l'alimentation (VCC et GND) au microcontrôleur ATtiny85.

2.1.2 En mode utilisation :

On débranche l'USBasp et on place le cavalier sur J4 pour alimenter le microcontrôleur via l'alimentation externe. Les cavaliers placés sur J 3 permettent de configurer les connexions SPI pour une éventuelle utilisation de l'ATtiny85 dans une application SPI avec d'autres composants.

2.2 Explication des composants essentielles du schéma

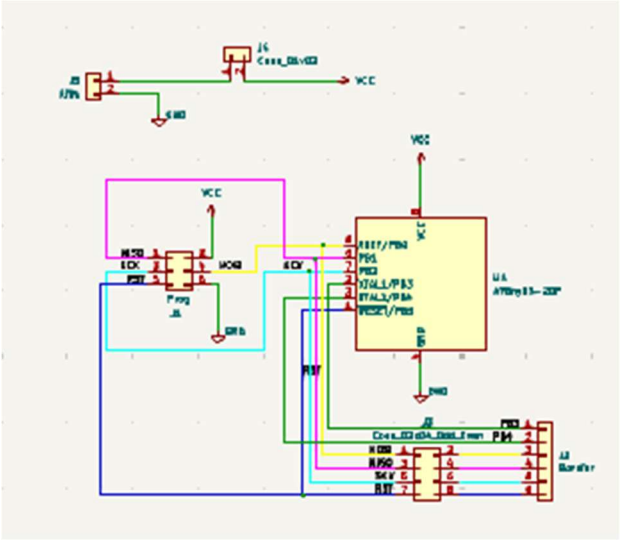
ATtiny85 (U1) : Le microcontrôleur est au centre du schéma. Il est alimenté par VCC et GND. Ses broches de communication SPI (MOSI, MISO, SCK) et RESET sont connectées pour permettre la programmation.

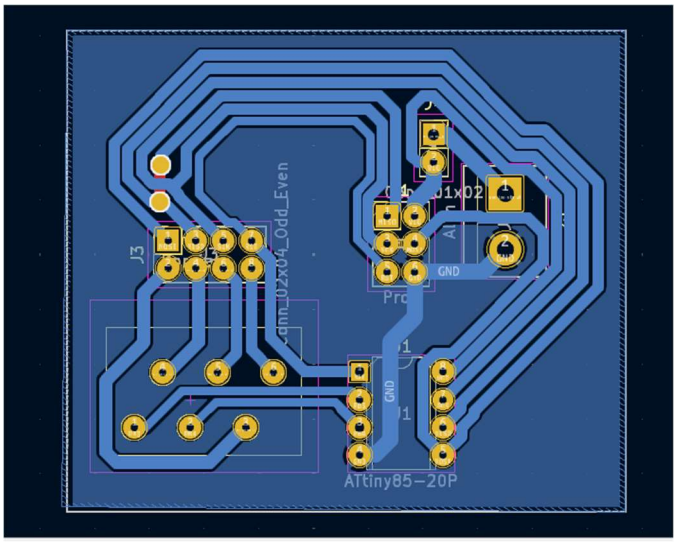
Connecteur de programmation (J1) : Ce connecteur (Prog) est destiné à accueillir un programmateur USBasp pour charger le code sur l'ATtiny85.

Alimentation (J4 et J5) : Ces connecteurs permettent d'alimenter le circuit :

J4 : Permet de relier VCC à l'ATtiny85. Ce cavalier doit être en place pour fournir l'alimentation en mode d'utilisation.

Cavalier (J3) : Ce cavalier permet de connecter les broches MOSI, MISO, SCK, et RST au bornier(J2), composants externes pour des applications SPI. En mode d'utilisation, les cavaliers sont placés ici pour établir les connexions nécessaires et il est débranché lors de programmation.





Voir schéma et routage

3 Programmation de l'ATTiny

J'ai fait un code qui utilise un microcontrôleur ATTiny85 pour lire des données numériques via une broche d'entrée série (MISO) et contrôler deux LED en fonction des valeurs binaires reçues.

3.1 Configuration des broches

Pour que le système fonctionne correctement, j'ai dû configurer les broches du microcontrôleur ATTiny85 en fonction de leur rôle. Voici comment cela a été fait :

- Les broches **PB0** (pour activer le composant externe, signal CS), **PB2** (pour l'horloge), **PB3** (LED verte) et **PB4** (LED rouge) ont été configurées comme des sorties, ce qui signifie qu'elles envoient des signaux.
- La broche **PB1** (MISO), en revanche, a été configurée comme une entrée, car elle sert à recevoir les données sous forme binaire.

Une fois les broches définies, j'ai initialisé leur état pour préparer le système :

- Le signal **CS** a été réglé à l'état haut, ce qui indique que le composant externe (le MCP3201) n'est pas encore actif.
- La broche **Horloge** a été mise à l'état bas pour démarrer proprement les échanges.
- Enfin, les LEDs (verte et rouge) ont été éteintes par défaut pour éviter toute confusion lors de la mise sous tension.

```c
int main(void) {
    // Configuration des broches
    DDRB |= (1 << DDB0); // PB0 - CS (Chip Select) en sortie
    DDRB |= (1 << DDB2); // PB2 - Horloge (CLK) en sortie
    DDRB &= ~(1 << DDB1); // PB1 - MISO (entrée des données)
    DDRB |= (1 << DDB3); // PB3 - LED VERTE
    DDRB |= (1 << DDB4); // PB4 - JAUNE

    // Initialisation des sorties
    PORTB |= (1 << PORTB0); // CS initialisé à l'état haut
    PORTB |= (1 << PORTB2); // Horloge initialisée à l'état bas
    PORTB &= ~(1 << PORTB3); // LED PB3 éteinte
    PORTB &= ~(1 << PORTB4); // LED PB4 éteinte
```

3.2 Boucle principale

Le fonctionnement du système repose sur une séquence précise de lecture des données et d'interaction avec les composants. Voici comment cela se déroule :

- Une variable appelée **donne_numerique** est utilisée pour stocker les données binaires reçues.
- Le processus commence par définir le signal **CS** à l'état haut (désactivation du composant externe) et l'horloge à l'état bas, suivi d'une courte pause de 10 millisecondes pour garantir une initialisation stable.
- Ensuite, **CS** est mis à l'état bas pour activer le composant externe et permettre la lecture des données.

La lecture elle-même se fait en plusieurs étapes :

- Une boucle de **15 cycles** est utilisée pour lire les données binaires de la broche **MISO** (l'entrée). À chaque cycle, l'horloge bascule d'un état bas à un état haut, permettant ainsi la lecture d'un bit. Chaque bit lu est ajouté progressivement à la variable **donne_numerique**.
- Une fois les 15 bits lus, **CS** est remis à l'état haut pour indiquer que la communication est terminée.

Ce processus, bien que précis et méthodique, m'a appris à manipuler avec soin les composants électroniques. Par exemple, lire les bits un par un et construire progressivement la donnée

complète m’a rappelé l’assemblage d’un puzzle, où chaque pièce compte pour obtenir une image complète. Cette étape m’a permis de mieux appréhender la manière dont les microcontrôleurs "parlent" aux autres composants, et de rendre cette communication fluide et fiable.

ec_arduino.ino

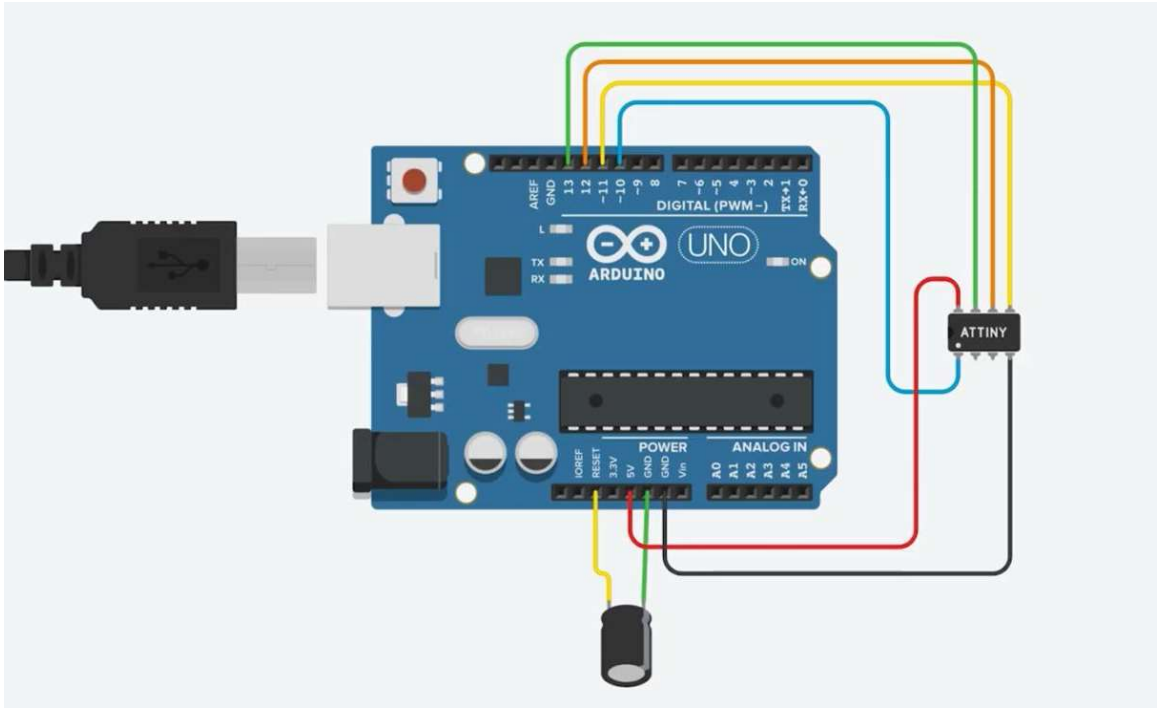
```
while (1) {
    uint16_t donne_numerique = 0; // Variable pour stocker les données binaires

    // Étape 1 : Mettre CS à l'état haut et Horloge à l'état bas
    PORTB |= (1 << PORTB0); // Mettre PB0 (CS) à l'état haut
    PORTB &= ~(1 << PORTB2); // Mettre PB2 (Horloge) à l'état bas
    _delay_ms(10);          // Attendre 10 ms

    // Étape 2 : Mettre CS à l'état bas
    PORTB &= ~(1 << PORTB0); // Mettre PB0 (CS) à l'état bas
    // Faire 15 impulsions
    for (int i = 0; i < 15; i++) {
        PORTB &= ~(1 << PORTB2); // Mettre PB2 (Horloge) à l'état bas
        _delay_us(60);           // Attendre 60 µs

        // Lire un bit à partir de PB1 (MISO) et l'ajouter dans `donne_numerique`
        donne_numerique = (donne_numerique << 1);
        if (PINB & (1 << PINB1)) {
            donne_numerique |= 1; // Ajouter cette valeur
        }

        PORTB |= (1 << PORTB2);  // Mettre PB2 (Horloge) à l'état haut
        _delay_us(60);           // Attendre 60 µs
    }

    PORTB |= (1 << PORTB0); // CS état haut
```

3.3 En fonction de la valeur de donne_numerique

- Si donne_numerique est supérieur ou égal à 0b011001100110 (qui représente environ 2V), la LED jaune (PB4) est allumée.
- Si donne_numerique est supérieur à 0b011001100110 mais inférieur à 0b110011001100 (qui représente environ 4V), la LED verte (PB3) est allumée.
- Si donne_numerique est supérieur à 0b110011001100 (plus de 4V), les deux LED (PB3 et PB4) sont allumées.

```
        }

        PORTB |= (1 << PORTB0); // CS état haut

        // Éteindre les LEDs par défaut (initialement)
        PORTB &= ~(1 << PORTB3); // Éteindre LED sur PB3
        PORTB &= ~(1 << PORTB4); // Éteindre LED sur PB4

        // Allumer les LEDs en fonction des valeurs binaires
        if (donne_numerique >= 0b011001100110) // Correspond à 2V
        {
            PORTB |= (1 << PORTB4); // Allumer LED sur PB4
        }
        else if (donne_numerique > 0b011001100110 && donne_numerique < 0b110011001100) // Correspond à entre 0V et 4V
        {
            PORTB |= (1 << PORTB3); // Allumer LED sur PB3
        }
        else if (donne_numerique > 0b110011001100) // Correspond à une tension superieue à 4V
        {
            PORTB |= (1 << PORTB3); // Allumer LED sur PB3
            PORTB |= (1 << PORTB4); // Allumer LED sur PB4
        }
    }

    return 0;
```

Schéma pour la programmation de ATTiny avec Arduino



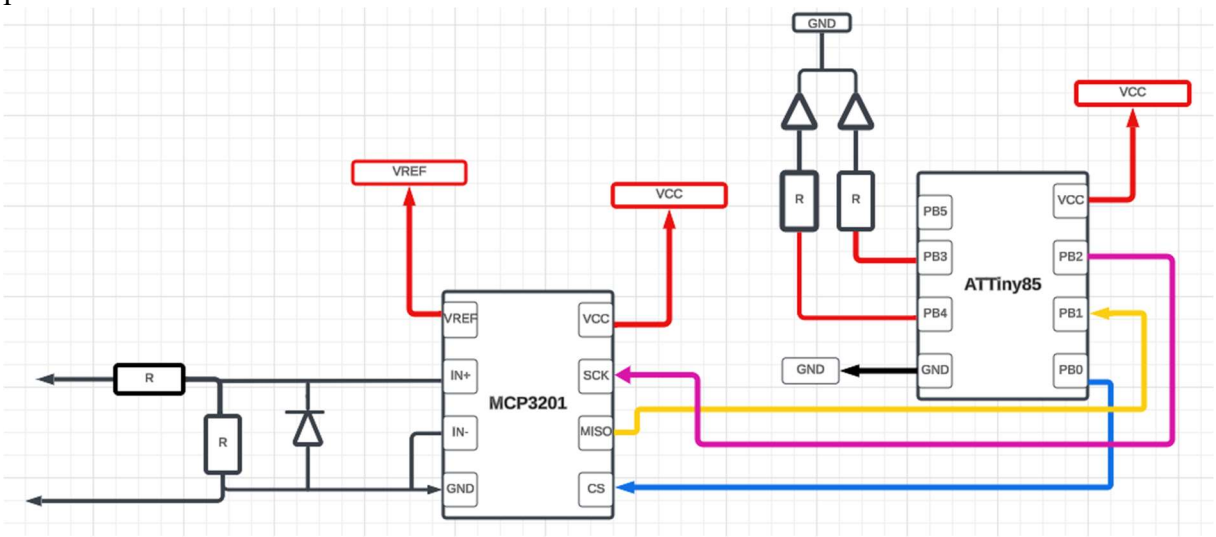
4 Connexion de l'ensemble (MCP et ATTiny) pour allumer les LEDs

Juste avant de faire cette partie, j'ai visualisé le CS et le Clock's (Horloge) sur oscilloscope.

Canal 1 (Jaune) : Ce canal montre le signal **CS (Chip Select)**. On peut voir qu'il passe de l'état haut à l'état bas, ce qui indique que le composant externe est activé et prêt à communiquer.

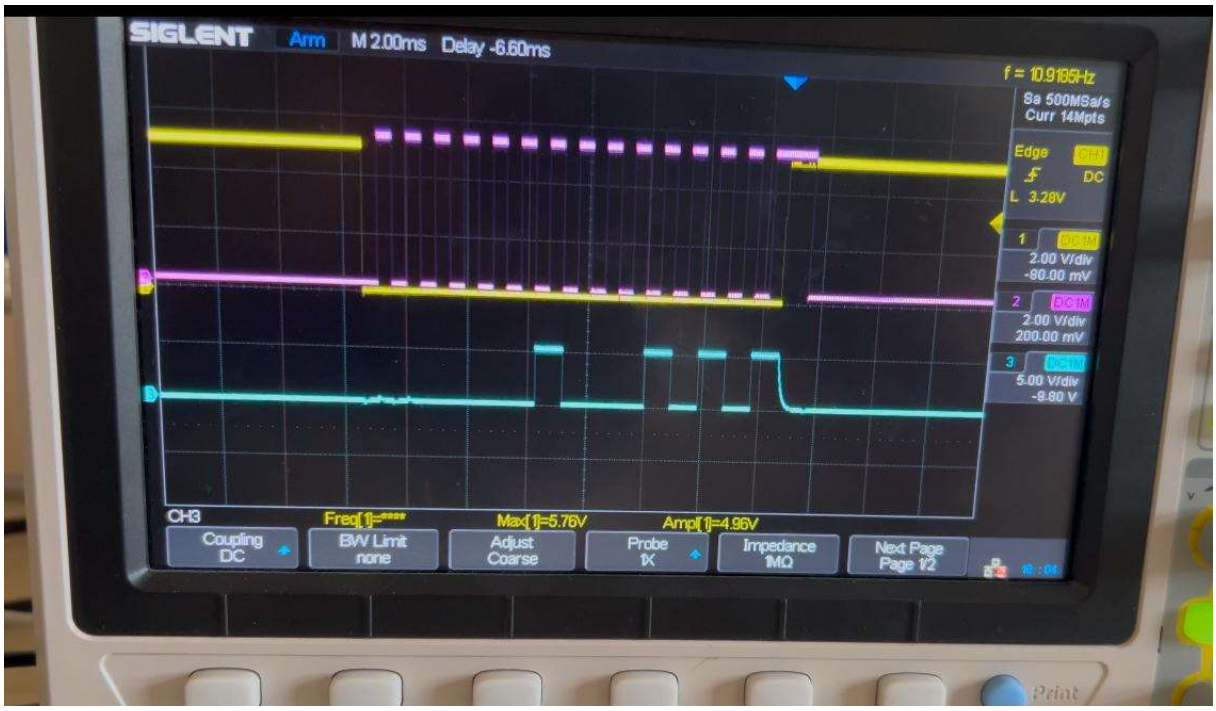
Canal 2 (Magenta) Ce canal représente le signal **Horloge (Clock)**. Il affiche une série d'impulsions régulières, typiques d'un signal d'horloge utilisé pour synchroniser les échanges de données.

L'analyse de ces signaux montre que l'horloge fonctionne correctement et reste active uniquement lorsque le signal CS est à l'état bas. Cette bonne synchronisation entre les deux signaux est un élément crucial pour garantir une communication série fiable, comme dans le protocole SPI.



Ce câblage permet de mesurer une tension comprise entre 0 et 14 V, de la convertir en valeur binaire grâce au MCP3201, puis d'envoyer ces données à l'ATtiny85. L'ATtiny85 utilise ensuite ces données pour contrôler l'allumage des LEDs en fonction des niveaux de tension.

La prochaine étape consiste à s'assurer que les données échangées entre les composants sont bien alignées avec ces signaux. Pour cela, il est nécessaire de câbler le circuit suivant et d'observer les données envoyées par le MCP3201 à l'aide d'un outil d'analyse, comme un oscilloscope ou un analyseur logique.



5 Compétences développées au cours de cette SAE

Dans le cadre du projet de conception du stylo voltmètre, j'ai appliqué et développé des compétences fondamentales qui s'inscrivent pleinement dans les objectifs du parcours **Électronique et Systèmes Embarqués (ÉSE)**.

Concevoir : Le projet m'a permis de travailler sur l'analyse des besoins pour définir des spécifications fonctionnelles adaptées, telles que la mesure de tensions continues et l'indication par LEDs. La sélection judicieuse des composants, comme le **MCP3201** et l'**ATTiny85**, ainsi que la création de circuits d'interfaçage, ont illustré ma capacité à concevoir des solutions robustes répondant à des contraintes techniques spécifiques.

Intégrer : La mise en œuvre de l'interface SPI pour assurer la communication entre les différents composants a été une étape clé. Ce travail a démontré ma capacité à intégrer des systèmes

électroniques complexes, comme cela est souvent requis dans des domaines tels que la domotique, la robotique ou les objets connectés.

Vérifier : La validation du système, notamment grâce à l'utilisation d'un oscilloscope, m'a permis de garantir la synchronisation des signaux et le bon fonctionnement du dispositif. J'ai également mis en place un protocole de tests rigoureux pour vérifier la fiabilité des mesures et l'alignement des données, une compétence essentielle pour des systèmes embarqués critiques dans des secteurs tels que l'aéronautique, la santé ou les transports.

Maintenir : La conception d'un support de programmation polyvalent a facilité la transition entre les phases de développement et d'utilisation. Cette solution modulaire démontre ma capacité à anticiper les besoins futurs en maintenance et à simplifier les interventions techniques.

Ce projet, bien qu'à échelle réduite, illustre les principes fondamentaux des systèmes embarqués, à savoir leur autonomie, leur capacité à interagir avec leur environnement, et leur intégration autour d'un microcontrôleur programmable. De plus, il reflète l'approche pratique et méthodique que le parcours **ÉSE** valorise pour répondre aux exigences de secteurs industriels variés comme la domotique, l'agriculture connectée ou encore l'intelligence artificielle.

Conclusion

Le développement de ce stylo voltmètre a été une expérience à la fois technique et pratique très enrichissante. Ce projet m'a permis d'approfondir mes compétences en électronique, tant analogique que numérique, ainsi qu'en programmation embarquée, et d'explorer la communication SPI entre le convertisseur analogique-numérique **MCP3201** et le microcontrôleur **ATTiny85**.

Grâce à l'utilisation d'un **diviseur de tension**, j'ai pu étendre la plage de mesure du MCP3201 pour mesurer des tensions allant jusqu'à 14 volts, tout en préservant la précision de ses mesures avec une résolution de 12 bits. L'ajout d'une protection contre les surtensions a également permis de sécuriser le circuit en cas d'anomalies.

La programmation de l'**ATTiny85** a été au cœur du projet, en traitant les données numériques envoyées par le MCP3201 et en contrôlant les LEDs pour afficher les différentes plages de tension mesurées. Les signaux **CS** et **Clock** ont été soigneusement synchronisés, et leur validation avec un oscilloscope a assuré une communication SPI fiable et stable.

Ce projet a présenté plusieurs défis techniques, notamment la configuration précise des signaux SPI, la gestion des données binaires, ainsi que la conception d'une interface visuelle simple mais efficace via les LEDs. Il m'a aussi sensibilisé à l'importance de la validation expérimentale pour garantir la fiabilité d'un système électronique.